

循序渐进，学习开发 xv6-riscv

第 0 章 开篇

汪辰



目录

01

课程介绍

02

操作系统简介

03

xv6 介绍

04

课程安排



01

课程介绍

课程简介

- 本课程是《循序渐进，学习开发一个 RISC-V 上的操作系统》的进阶版本。

循序渐进，学习开发一个
RISC-V 上的操作系统



扫码听课

前置课程代码仓库：

<https://gitee.com/unicornx/riscv-operating-system-mooc>

在线答疑QQ群和微信群（加入方式具体看 Github/Gitee 仓库的 README 文件）

- 本课程的代码参考自 xv6-riscv。
- 本课程代码仓库: <https://gitee.com/unicornx/xv6-riscv-step-by-step>。更全面的课程信息请查看课程代码仓库的 *README_cn.md* 文件。

学习本课程的目的

“讲到这里，我想对青年学生们说几句话，不管你们将来从事什么职业、有什么样的志向，一定要注意加强基础知识学习，打牢基本功和培育创新能力是并行不悖的，树高千尺，营养还在根部。把基础打牢了，将来就可以触类旁通，行行都可以写出精彩。”

——摘自十三届全国人大四次会议记者会
国务院总理李克强讲话文字实录

学习本课程的目的

- 学习开发一个 UNIX-like 的，基于 RISC-V 架构的操作系统。
并在此过程中：
- 学习一些操作系统的基本概念。
- 掌握一些基本的底层架构相关的开发技能。
- 学习和了解一些 RISC-V 的相关知识。

学习本课程的前置要求

- 比较熟练地掌握 C 语言编程。
- 对 RISC-V 有一定了解并相对熟练地掌握 RISC-V 汇编编程。
- 掌握基础的数据结构开发（基于 C 语言）。
- 了解 Linux 上开发和调试的基本知识。

如果对以上一条或多条感觉掌握程度不足的，建议先学习本课程的前置课程

循序渐进，学习开发一个
RISC-V 上的操作系统



扫码听课

前置课程代码仓库：

<https://gitee.com/unicornx/riscv-operating-system-mooc>

在线答疑QQ群和微信群（加入方式具体看 Github/Gitee 仓库的 README 文件）



02

操作系统简介



操作系统是什么

操作系统（英语：Operating System，缩写：OS）是一组主管并控制计算机操作、运用和运行硬件、软件资源和提供公共服务来组织用户交互的相互关联的系统软件程序，同时也是计算机系统的内核与基石。

摘录自维基百科

操作系统的核心能力

- 虚拟化 (Virtualization)
- 并发 (Concurrency)
- 持久性 (Persistence)



操作系统的核心能力

- 虚拟化 (Virtualization)

所谓虚拟化，是指操作系统通过虚拟化技术，将物理资源转化为 *可隔离、可灵活分配的逻辑资源*。

- CPU 虚拟化
- 内存虚拟化
- 并发 (Concurrency)
- 持久性 (Persistence)

操作系统的核心能力

- 虚拟化 (Virtualization)

- 并发 (Concurrency)

操作系统需要保证在*多处理器并行*、*线程切换*或*中断*所导致的多个指令流交错执行的情况下代码逻辑的正确性。

- 持久性 (Persistence)

操作系统的核心能力

- 虚拟化 (Virtualization)
- 并发 (Concurrency)
- 持久性 (Persistence)

确保数据可以 *持久* 地保存，包括提供一套抽象的 *统一访问接口*。



03

xv6 介绍

xv6是什么

xv6 操作系统是由麻省理工学院（MIT）的 PDOS 实验室为教授操作系统课程而开发的教学用操作系统。

It explains the main concepts of operating systems by studying an example kernel, named xv6. Xv6 is modeled on Dennis Ritchie's and Ken Thompson's Unix Version 6 (v6). Xv6 loosely follows the structure and style of v6, but is implemented in ANSI C for a multi-core RISC-V.

——摘自《xv6: a simple, Unix-like teaching operating system》前言

xv6是什么

xv6 操作系统是由麻省理工学院（MIT）的 PDOS 实验室为教授开发的教学用操作系统。



<https://www.bilibili.com/video/BV1py4y1y7wD/>

main concepts of operating systems by studying an unnamed xv6. Xv6 is modeled on Dennis Ritchie's and Unix Version 6 (v6). Xv6 loosely follows the structure and style of v6 but is implemented in ANSI C for a multi-core RISC-V.

《Unix-like teaching operating system》前言

xv6 的发展历史

纪年	大事记
2002	MIT 开设经典课程“6.828: Operating System Engineering”，但早期的实验平台使用的是其自研的 JOS 操作系统（基于 x86，用 ANSI C 编写）。
2006	Russ Cox、Frans Kaashoek、Robert Morris 等教授开发了初版 xv6（基于 x86 架构），复刻了 UNIX V6 的核心思想，并首次用于 MIT 6.828 的课程实验。
2017	MIT PDOS 实验室从简化教学的目的出发，启动了移植工作，开始将 xv6 从 x86 迁移到 RISC-V（64 位）。
2019	MIT PDOS 重组了课程体系，将原 6.828 拆分为“6.S081: Operating System Engineering”和 6.828（进阶课程，包括介绍分布式和安全等专题），此时 xv6 成为 6.S081 唯一的实验和教学系统，并全面转向 RISC-V 架构，
2020	全面过渡完成，原 x86 版本停止功能更新，仅维护历史版本，所有新特性仅在 RISC-V 版本上开发
2022	6.S081 又更名为 6.1810 并沿用至今。

xv6 实现了哪些功能

- 支持 RISC-V 64 位处理器，并基于模拟器 QEMU 的 virt 机器运行。
- 支持多核 SMP 架构。
- 支持用户态和内核态的划分，内核运行在内核态，用户程序运行在用户态。
- 实现了基于时间片的、抢占式的、多任务轮转的调度策略。
- 支持进程的概念，每个进程拥有独立的虚拟地址空间。
- 支持简单的进程间通讯，譬如管道。
- 参考 UNIX，实现了 21 个系统调用。
- 支持基本的并发控制，包括自旋锁和睡眠锁。
- 支持带日志的文件系统，支持 UNIX-like 形式的文件和目录层次结构。
- 实现了一些简单的外设驱动，支持串口和符合 virtio 标准的磁盘设备。
- 作为一个相对完整的操作系统，xv6 还实现了一些运行在用户态的应用程序。

xv6 没有实现哪些功能

- 没有实现用户的账号管理功能，也不支持用户登录。可以认为 xv6 在运行时只有一个用户 root。
- 没有实现文件的保护，譬如文件权限检查，读、写以及可执行权限控制等。
- 只有一个根文件系统，不支持类似 mount 或者 umount 操作来挂载其他的文件系统。
- 不支持 SWAP 操作，即不支持在内存紧张时利用磁盘对内存页进行交换的功能。
- 不支持多线程，虽然支持了多进程但每个进程中只有一个线程，这也是早期 UNIX 系统的经典进程实现方式。
- 不支持网络，即没有实现现代操作系统中都具备的 socket 接口。
- 只实现了简单的管道操作，不支持复杂的进程间通讯机制，譬如消息队列等。
- 对设备驱动的支持有限，现实操作系统内核的几百万行代码中有百分之七十以上都是驱动相关的代码，当然 xv6 作为一个教学系统，也的确不需要实现那么多的外设。
- 只实现了很小的一部分应用程序。

xv6 系统调用一览

```
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int*);
int kill(int);
int exec(const char*, char**);
int getpid(void);
char* sys_sbrk(int, int);
int pause(int);
int uptime(void);
```

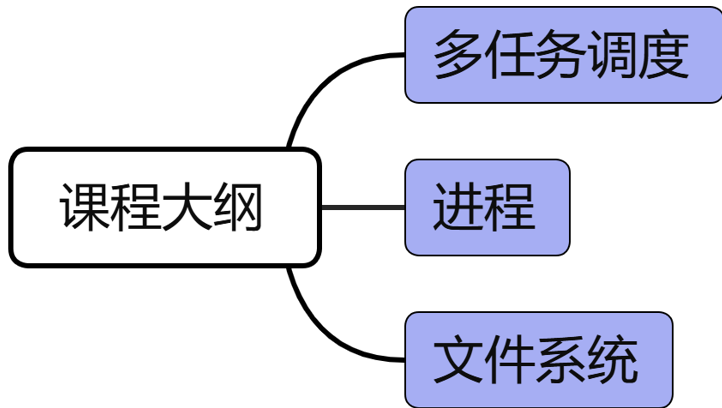
```
int open(const char*, int);
int read(int, void*, int);
int write(int, const void*, int);
int close(int);
int unlink(const char*);
int fstat(int fd, struct stat*);
int link(const char*, const char*);
int mkdir(const char*);
int chdir(const char*);
int dup(int);
int mknod(const char*, short, short);
int pipe(int*);
```



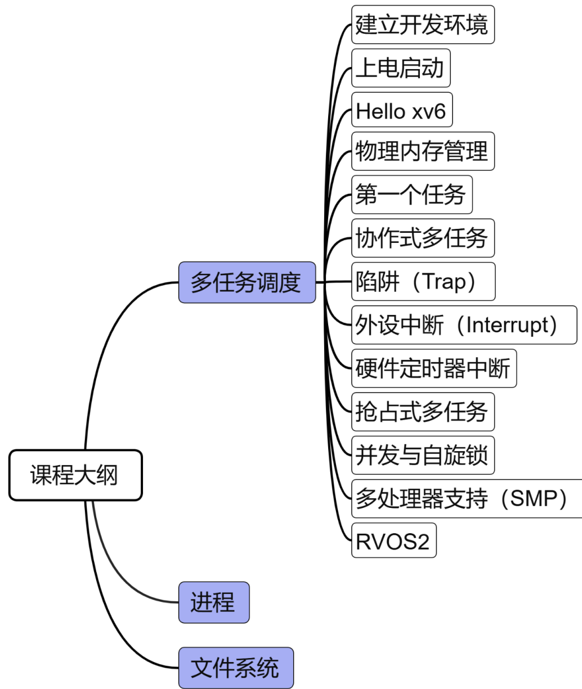
04

课程安排

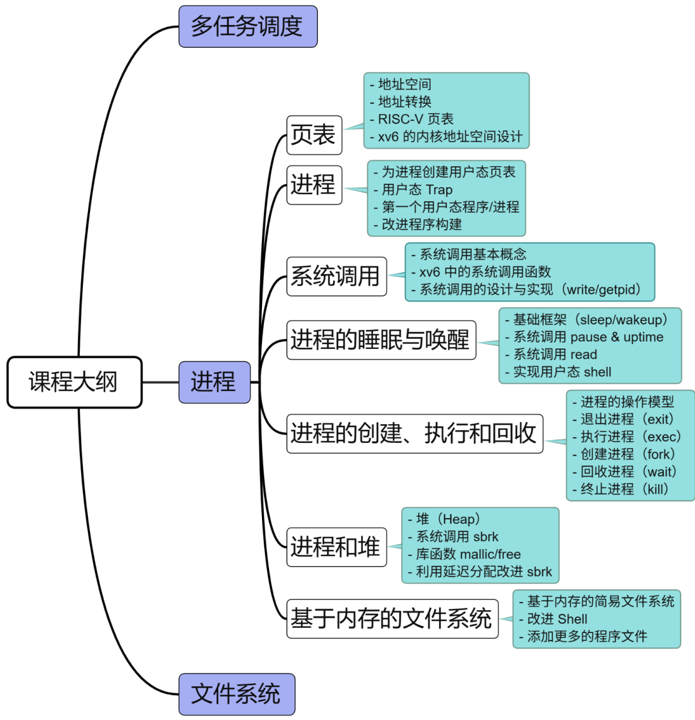
课程大纲



课程大纲



课程大纲



谢谢

